

以 AlphaZero 方法實現 Connect6 強化學習系統

李承遠

Department of Computer Science and
Information Engineering
National Chi Nan University
Nantou, Taiwan
s111321004@mail1.ncnu.edu.tw

張岷凱

Department of Computer Science and
Information Engineering
National Chi Nan University
Nantou, Taiwan
s111321062@mail1.ncnu.edu.tw

周信宏

Department of Computer Science and
Information Engineering
National Chi Nan University
Nantou, Taiwan
chouhh@ncnu.edu.tw

Abstract—本專案基於 AlphaZero 框架為遊戲 Connect6 實現了一個強化學習系統，並集成了 Monte Carlo Tree Search (MCTS) 和深度卷積神經網路。此專案適應了 Connect6 的特別規則，包括每回合下兩子的機制和連續形成六子的勝利條件。訓練資料使用 MCTS 探索通過自我對下生成，並且通過使用生成的策略和價值目標通過監督學習來優化模型。

我們的專案以 Alphazero 嘗試各種組合訓練六子棋專用模型。本專案運用 AlphaZero General 強化學習框架實作六子棋，透過自我對弈、蒙地卡羅樹搜尋 (MCTS) 與深度神經網路的結合，使代理模型能在無需人類資料或專家知識的情況下自行學習下棋策略。專案中將六子棋規則實作為相容於 AlphaZero General 的自訂遊戲環境，使模型能透過不斷的自我對弈與訓練循環，逐步優化其策略與局面評估能力。

Keywords— 六子棋、強化學習、AlphaZero、蒙地卡羅樹搜尋、深度學習

I. INTRODUCTION

人工智慧在棋類遊戲領域的突破性進展，始於 2016 年 Google DeepMind 團隊[3]發表的 AlphaGo 系統。AlphaGo 結合深度神經網路與蒙地卡羅樹搜尋 (Monte Carlo Tree Search, MCTS)，透過圍棋規則與人類對局資料進行學習，成功擊敗世界圍棋冠軍李世石，展示了 AI 在專業策略遊戲中的潛能。其後於 2017 年推出的 AlphaGo Zero[4]，不同於傳統棋類程式仰賴人工設計的搜索技巧和領域知識，AlphaZero 運用深度神經網路和 Monte-Carlo 樹搜索，從零開始學習圍棋、象棋和將棋。證明了 AI 系統能在無人工指導下自主學習並優化策略。

Connect6 是一款源自 Gomoku 的複雜雙人棋盤遊戲，發明人國立交通大學資訊工程系教授吳毅成為改良自五子棋一般規則及連珠棋規中「先手必勝」之不合理情況及國際棋規中之繁複規則。其規則是黑棋下第一步后，每個玩家每回合必須放置兩顆棋子。與 Gomoku 相比，Connect6 的動作空間明顯更大，這使得傳統演算法更難有效評估遊戲狀態。為了應對這一挑戰，我們的項目採用了 AlphaZero 框架，該框架將蒙特卡洛樹搜索 (MCTS) 與深度卷積神經網路相結合，通過強化學習訓練自我改進的模型。

楊子頤[1]於國立陽明交通大學資訊工程研究所提出之碩士論文《應用 AlphaZero 於六子棋》。在系統實作方面，作者使用交通大學 CGILab 團隊開發之 AlphaZero 軟體框架 CZF18，搭配 15 層殘差網路 (192 filters) 進行訓練，訓練時間約 21 天，共動用 21 張 GTX 1080 Ti GPU，自我對弈產生約 200 萬局遊戲資料。初期也輔以專家棋譜進行引導，提升模型初始品質。該研究亦深入探討「邊界問題」，指出因自我對弈過程大多集中於棋盤中央，導致邊緣威脅判斷能力不足。透過主動加入邊界盤面訓練資料，顯著改善模型的邊界防守能力。

此外，該論文提出一種 turn-based 下棋策略，根據 MCTS 結果於每回合一次選擇兩顆子，相較於原始逐顆選擇 (stone-based) 方式，在短思考時間內仍能維持更高的棋力。研究亦探討不同深度與寬度的 ResNet 結構對表現的影響，發現增加 Residual Block 數量能有效提升訓練效果。最終，該 AlphaZero 六子棋程式於與先前開發之 NCTU6 程式對戰中取得 74% 勝率，並在 Little Golem 平台上獲得年度冠軍與十次月賽冠軍，實驗證實其訓練成效與競技實力。

在林育璋[2]的研究中，作者針對陽明交通大學所提出的通用型 Bitboard 架構進行修改與優化，並應用於六子棋的 AlphaZero 強化學習系統。在效能提升方面，透過 C++ 重寫與平行化蒙地卡羅樹搜尋（Monte Carlo Tree Search, MCTS），並搭配 Virtual Loss 避免執行緒重複探索，使得搜尋速度提升 26.4 倍。同時結合批次神經網路預測與 GPU 加速，以及將訓練與資料產生階段並行執行，進一步提升整體效率。

作者也同時強化了神經網路性能，他借鏡了 KataGo，Global Pooling 架構與 Auxiliary Policy Targets 技術，並針對六子棋雙子落子的特性，設計能預測對手兩步棋的輔助 policy head。並且為了在訓練過程優化訓練資料品質，使用 Forced Playout、Policy Target Pruning 與五子棋啟發式策略中的 Threat Space Search（TSS）提升資料品質。特別是保守型 TSS 結合 Bitboard 實作達到 6.03 倍加速，並用於尋找必勝解。論文中藉由 ELO 測試與對戰驗證，證實這些方法在找必勝能力與總體棋力上皆優於對照模型。

兩篇研究分別從「演算法優化與啟發式結合」以及「規則改寫與實作訓練」兩個層面切入，互為補充，對於本研究中 AlphaZero 六子棋系統的設計與改進提供了實作基礎。我們的目標是基於 AlphaZero 構建一個功能齊全的 Connect6 訓練管道，並對其進行修改以處理兩子的動作空間。我們實現了遊戲邏輯、具有可調整類比的 MCTS，以及使用自我對戰數據訓練的深度殘差網路。本研究的貢獻主要有兩點，將 AlphaZero 架構成功應用於傳統演算法策略低效的六子棋領域，以及在運算資源有限的條件下，觀察並分析模型的學習過程，為未來於個人電腦上訓練大型棋盤類強化學習系統提供可行性與實作經驗。

II. METHODS

本研究所實作的 Connect6 強化學習系統，整體架構以 AlphaZero 為框架，融合蒙地卡羅樹搜尋與深度神經網路技術，並加入支援六子棋規則的客製化邏輯。透過自我對弈產生資料、以策略分布（policy distribution, π ）與盤面評估值（value, v ）進行訓練，並持續迭代模型以提升棋力。

A. AlphaZero 系統架構

AlphaZero 是一種基於自我對弈學習（self-play reinforcement learning）並結合蒙地卡羅樹搜尋（Monte Carlo Tree Search, MCTS）與深度神經網路的通用強化學習架構。其最大特色是完全不依賴人類知識或棋譜，自主從零學習並迭代強化策略。本系統即仿照 AlphaZero 的設計，應用於 Connect6 棋類環境，並依據雙子落子的特殊規則進行相應調整。以下分別說明系統所採用的核心模組與運作流程。

1) 蒙地卡羅樹搜尋（MCTS）

MCTS 是 AlphaZero 中策略探索與決策選擇的基礎。在每一回合，當模型須決定落子位置時，MCTS 將從當前棋盤狀態 s 展開模擬若干局勢，並依據神經網路提供的策略分布 π 與盤面價值 v 擴展節點與反向傳遞資訊。

系統會為每個棋盤狀態維護如下統計資訊： $Ps[s]$ （由神經網路預測之初始策略分布）、 $Qsa[s,a]$ （狀態 s 與行動 a 的平均價值）、 $Nsa[s,a]$ （行動 (s,a) 被模擬的次數）、 $Ns[s]$ （整個狀態 s 被模擬的總次數）。在每次搜尋中，MCTS 選擇擁有最大上信賴界（Upper Confidence Bound, UCB，用以平衡探索與利用）的節點向下展開，直到抵達葉節點為止。葉節點將呼叫神經網路預測 π 與 v ，再將 v 作為評估值沿路反向傳遞至根節點。最終，MCTS 將產生一個動作機率向量 π ，作為落子選擇依據，亦可於實際對局中用以隨機選擇落子，促進探索。

2) 策略與價值網路（Policy and Value Network）

本系統使用一個改良版的 ResNet（Residual Network，具跳接結構的深度卷積網路）架構作為神經網路模型。其輸入為 19×19 單通道棋盤影像，輸出則分為兩條分支：一條為 policy head，輸出 softmax 機率分布；另一條為 value head，預測目前棋局對當前玩家的勝率估計值（範圍為 -1 至 $+1$ ）。網路主體由一個卷積層與多層殘差區塊（Residual Blocks）組成，每個區塊包含兩層卷積層與跳接（skip connection）。Policy Head 包含一層 1×1 卷積與 Dense 層；Value Head 則接續兩層全連接層與 Dropout（隨機遮蔽神經元以防過擬合），最終輸出一個 tanh 激活值作為勝率評分。損失函數如下方程式，其中 z 為實際

對弈結果（勝=1，敗=-1）， π 為 MCTS 所產生的策略標籤， p 為網路輸出策略，第三項為 L2 正則化項，避免模型過擬合。

$$l = (z - v)^2 - \pi^T \log p + c || \theta ||^2$$

3) 自我對弈與資料收集流程

訓練資料主要透過自我對弈（Self-Play）產生。在每輪訓練中，系統會使用多執行緒（multiprocessing）啟動多個平行 worker，呼叫 `self_play_worker()` 各自執行一場完整對弈，並於每一步收集樣本（board, π , v ）。其中 board 為以當前玩家角度表示的棋盤狀態， π 為該步由 MCTS 推導出的機率分布， v 為最終勝負結果對當前玩家的結果（勝為 1、敗為 -1），所有樣本將經由資料增強處理，包括旋轉與鏡射棋盤與 π 向量後進行擴充，最終送入神經網路進行訓練。

4) 模型更新與對戰評估機制

每輪訓練後，系統會執行一個稱為「Arena」（對戰場，用於新舊模型比較）的對戰評估程式，由新訓練出的模型與上一輪模型進行固定場數的對戰，交叉交換先後手。若新模型勝率超過預設門檻，將更新模型為新的 best.h5，否則保留舊版本。避免模型因短期波動退步。

B. 主控制流程（mainConnect6.py）

這個模組是整個系統的起點，主要負責初始化訓練過程所需的各種參數，包括訓練的輪數、自我對弈的場數、使用的 CPU 核心數、以及 MCTS 模擬的次數等。此外，它也支援載入之前訓練好的模型來續跑，支援長時間訓練。Fig. 1. 為主程式運作之流程圖，主程式 `mainConnect6.py` 為訓練起點，負責建立訓練參數與呼叫 `Coach` 模組進行自我對弈控制。`Coach` 中的核心邏輯會以 multiprocessing 啟動多 worker 平行執行對局。每場對弈過程透過 `Connect6Game.py` 的 API（如 `getInitBoard()`、`getNextState()`、`getValidMoves()`）操作棋盤，底層邏輯由 `Connect6Logic.py` 實作。所有對弈資料（棋盤、策略 π 、勝負結果 v ）蒐集完成後，交由 `NNetWrapper`（包裝於 `Connect6NNet.py`）進行神經網路訓練。訓練完成的新模型將進入 `ArenaConnect6.py` 進行與上一輪模型的對戰評估。若新模型勝率高於設定門檻，則更新為目前最強版本，並進入下一輪訓練；否則保留舊模型繼續訓練。此迭代機

制確保模型強度能穩定提升，避免退步或過擬合。

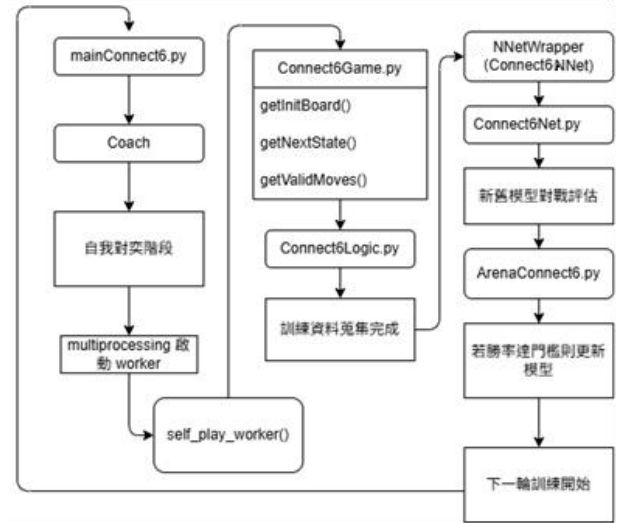


Fig. 1. 主程式之流程圖

C. 訓練流程控制模組

`Coach` 模組是整個系統的核心，負責統籌每一輪的自我對弈流程、資料蒐集以及模型更新。`Coach.py` 作為訓練的主要程式，會首先載入遊戲規則與神經網路，隨後進入自我對弈階段。在這個階段中，系統會將目前的棋盤狀態、神經網路以及遊戲規則傳遞給 `MCTS.py`，由 `MCTS` 的搜尋結果決定每一步的落子位置，直到完成預定的對弈場數，並且為了加快對弈資料的生成，我們有用 `multiprocessing` 技術來同時開啟多個子進程進行模擬。完成後，會將對弈過程中收集的資料儲存起來，最後透過 `NNet.py` 執行神經網路的訓練並保存更新後的模型權重。每一輪訓練完成後，我們會用 `ArenaConnect6.py` 進行新舊模型的對戰評估。如果新模型的勝率高於設定的門檻（像是 60%），我們就會把它更新成目前的最佳模型。

D. 六子棋遊戲邏輯設計（Connect6Game.py）

本系統所支援之 `Connect6`（六子棋）遊戲規則相較五子棋更為複雜，規定雙方玩家除黑子開局第一手僅下一子外，其餘每回合均須同時下一對子。勝利條件為棋盤上出現六顆連線（橫、直、斜任一方向）。為實現六子棋遊戲邏輯，我們實作了 `Board` 類別，`Board` 物件以 `NumPy` 陣列記錄盤面狀態，並支援放置棋子、回傳合法落子座標、檢查是否有連線勝利等功能。又為因應雙子落子規則，系統在每一回合使用 `flag` 判斷目前為第一手還是第二手，再決

定是否進行換手。遊戲的每一回合均可呼叫 getNextState 函式更新棋盤與玩家，並藉由 getValidMoves 回傳目前所有合法動作的 one-hot 向量（僅有一個位置為 1 的類別編碼方式）。

E. 蒙地卡羅樹搜尋模組 (MCTS.py)

蒙地卡羅樹搜尋模組 (MCTS.py) 是我們實作 AlphaZero 系統中的核心之一，主要負責在每一手棋中進行模擬與決策。它會根據目前棋盤狀態，結合神經網路輸出的策略分布 (π) 與局勢評估值 (v) 來導引搜尋方向。整個搜尋過程會從根節點開始，逐步選擇子節點直到抵達尚未展開的節點，接著根據神經網路的預測結果進行擴展與節點更新。每次模擬完成後，會將結果反向傳遞，更新路徑上每個節點的統計資訊，例如 Q 值（平均報酬）和訪問次數。經過多次模擬之後，模組會根據節點的訪問次數來決定實際落子的動作。這樣的設計能夠有效結合搜尋與學習，提升落子的合理性與整體棋力。

F. 神經網路架構模組 (Connect6Net.py)

神經網路架構模組 (Connect6Net.py) 是我們設計的棋盤狀態評估模型，整體架構是簡化版的殘差網路 (ResNet)。輸入為 19×19 的單通道棋盤矩陣，代表目前的對局狀態，經過一層卷積與數個殘差模組後，提取出深層棋盤特徵。這些特徵會被送入兩個輸出分支：其中 policy head 預測每一個位置的落子機率，用來輔助 MCTS 做出決策；而 value head 則預測目前棋盤的勝負機率，作為整體局勢的量化評估。兩個輸出頭會同時參與訓練，使模型既能學會合理選點，也能掌握局勢變化，有助於強化學習過程的穩定與收斂。

G. 模型對戰評估模組 (ArenaConnect6.py)

模型對戰評估模組 (ArenaConnect6.py) 為用於驗證訓練出來的新模型是否確實優於舊版本。此模組會讓新模型與先前的模型進行多場對戰，並採用輪流先手的機制以減少先手優勢的影響。透過統計對戰結果來計算新模型的勝率，若其表現優於舊模型（通常設有最低勝率門檻），就會將新模型保存為最新的標準，並用作後續的訓練與對戰基礎。

III. Experiment

A. 訓練參數及超參數設定

為有效訓練本系統之 AlphaZero 強化學習架構，並同時考量到個人電腦的運算資源限制，我們設計並設定了一系列訓練參數與超參數，涵蓋自我對弈的輪數、每場對局的模擬深度、模型訓練參數與資料記憶策略等，詳細設定參數如表 I 所示。其中，自我對弈訓練共進行 30 輪 (numIters)，每輪進行 25 場對局

(numEps)，每一步均使用 50 次 MCTS 模擬 (numMCTSSims)，以確保策略探索的深度。新模型與前一代模型之間以 10 場對戰 (arenaCompare) 評估勝率，若新模型勝率達 60% (updateThreshold)，則予以更新。此外，神經網路使用學習率 0.002、batch size 為 256、訓練 50 個 epoch，並透過 dropout 機制（比例 0.3）降低過擬合風險。

TABLE I. 實驗訓練參數與超參數設定表

參數名稱	訓練細節	
	參數意義	參數數值
numIters	總訓練輪數 (iterations)	30
numEps	每輪 self-play 對局數 (episodes)	25
tempThreshold	溫度變化的步數閾值 (值大會增加探索)	3
updateThreshold	新模型取代舊模型的勝率門檻	0.6
maxlenOfQueue	訓練資料最大長度 (樣本記憶)	100000
numMCTSSims	每步進行的 MCTS 模擬次數	50
arenaCompare	舊新模型對戰次數 (評估用)	10
cpuct	MCTS 中 UCB 機制的探索參數	1.0
numItersForTrainExamplesHistory	保留幾輪的訓練資料歷史	5
Learning_rate	模型學習率	0.002
dropout	防止過擬合	0.3
epochs	訓練輪次	50
batch_size	訓練批次大小	256
num_channels	通道數量	128

B. 自我對弈訓練過程

每輪訓練會使用 multiprocessing 啟動若干個 self-play worker，透過 MCTS 決定策略，進行模擬對弈。產生的資料樣本為 (board, pi, v) 格式，經資料增強（旋轉/鏡射）後送入模型訓練。Fig. 2.為截至第 30 個迭代時保留的前五個迭代的資料量，平均資料量為 36,358。

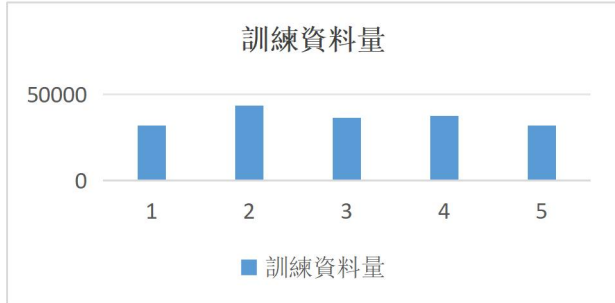


Fig. 2. 第 30 個迭代保存的前五個迭代資料數量

Fig. 3.為第 30 個迭代的訓練損失折線圖，從圖中可以清楚觀察到兩條曲線皆呈現穩定遞減趨勢，Policy Loss 從初始值約 2.8 穩定下降至接近 0.1，代表模型逐漸學會如何預測 MCTS 所給予的策略分布，而 Value Loss 則於初期迅速下降至約 0.1 以下並持續緩降，代表模型能從棋盤結構中學會預測當前局勢的勝負趨勢。

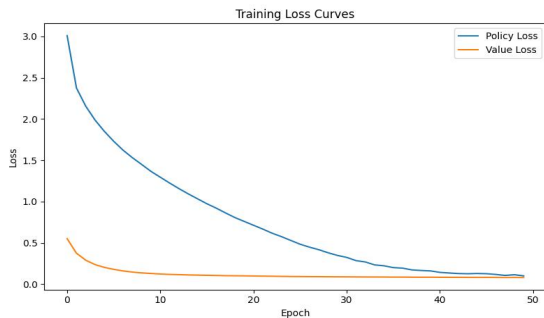


Fig. 3. 模型對於訓練資料之訓練損失曲線圖

C. 訓練結果

為確認新模型是否學得較佳策略，每輪訓練後都會進行一次 Arena 評估，由新模型與先前最佳模型進行 10 場對戰，其中勝率超過 60% 的模型將保存為最佳模型。Fig. 4.為訓練過程中模型勝率變化圖。

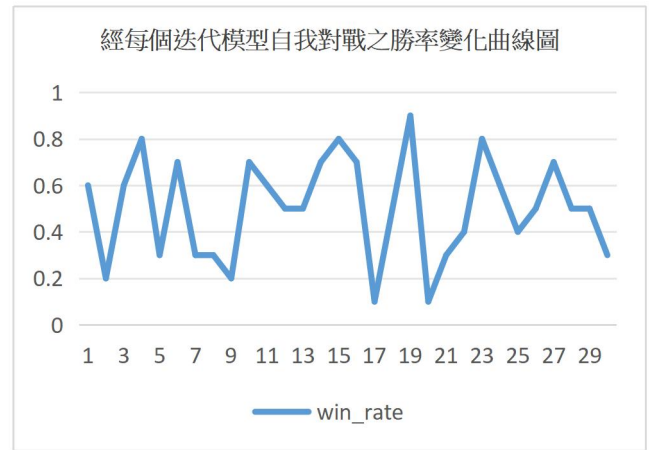


Fig. 4. 經每個迭代模型自我對戰之勝率變化曲線圖

D. 結果觀察

儘管本專題已完成初步的自我對弈與模型訓練流程，然而由於訓練成本與資源限制，目前尚僅產生約 30 局對弈、共計約 750 筆資料，相比於理論中能達到超人類水準的 200 萬場資料差距甚多。受限於資料量不足，與 MCTS 模擬次數偏低，僅模擬 50 次，模型尚未充分學習出如「連線」、「封鎖對手」、「中心優先」等策略，因此在實際盤面上尚未觀察到明顯的策略行為。

在目前模型的預測結果中，落子位置仍呈現偏隨機性，較常分布於空白區域的邊角或非關鍵位置，未能集中於形成連線或阻斷路徑的區域。從對局觀察亦可發現模型落子間缺乏連貫性，對於對手威脅反應不足，表示其尚未形成有效的局部攻防判斷機制。Fig. 5.為訓練資料中之關鍵局面，該盤面位於黑棋第一子回合，棋盤上在第 12 列上已有了連下兩子的連線威脅（下在位置 (4,12) 與位置 (6,12) 將可形成六連之局面）。然而，從模型輸出的 π 向量預測最佳步可見，其僅在第 0 列第 13 行（位置 (0,13)）上給出機率值 1，代表策略向量在其餘位置機率皆為 0。顯示模型對當前盤面的決策幾乎完全集中於一個非關鍵位置，並未選擇能直接達成勝利或封鎖威脅的落子點。

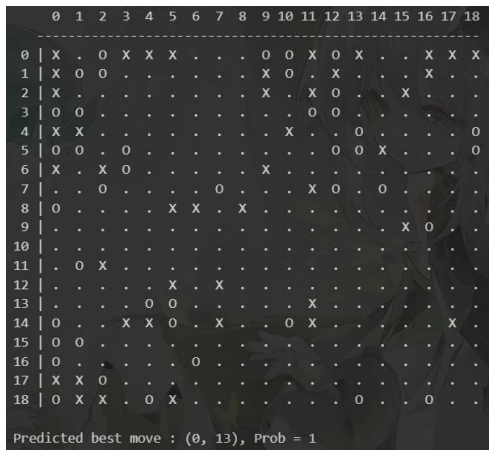


Fig. 5. 訓練資料中之關鍵局面

此現象反映出強化學習在高動作空間下的困難處，尤其在初期資料不足的情況下，神經網路暫時無法擷取有效策略特徵，導致在初始局面下 policy head 所學的機率分布趨近均勻，並且關鍵局面下 policy 的唯一落子並非必勝抉擇，間接導致了 value head 所估計的勝率亦缺乏穩定性。

IV. Conclusion and Future work

本研究建構了一個以 AlphaZero 為基礎架構的六子棋 (Connect6) 強化學習系統。透過結合蒙地卡羅樹搜尋 (Monte Carlo Tree Search, MCTS) 與深度神經網路，我們實現了自我對弈資料生成、自動化模型訓練與新舊模型對戰評估等完整流程。六子棋屬於動作空間極大的棋類遊戲，相較於五子棋或圍棋，其每一步可能的落子位置更多，且回合中一次落下兩子的特殊規則亦導致狀態空間爆炸。對於使用 MCTS 的演算法來說，在目前資源有限的環境下會帶來龐大的運算壓力與效率瓶頸。現階段實作版本雖能在個人電腦上順利運行，但在模擬次數提升或擴展模型規模時仍極大的受限於運算速度。

儘管如此，我們認為此階段仍具重要觀察價值。未來應嘗試更加優化專案效能，提升每局 MCTS 模擬次數、增加自我對弈輪數，並延長訓練週期後，模型應能逐步學習更穩定的策略。後續也將持續追蹤模型在連子策略、阻擋策略的應對、以及更傾向於在棋盤中心落子的行為。若後續模型出現戰略性行為時，預期將觀察到 π 向量集中於已有連線或連線缺口的延伸點或連線缺口，並且在對手連四或活三威脅出現時傾向選擇封鎖落點的位置。目前階段雖仍屬探

索初期，但已為後續完整學習流程奠定架構基礎，並可透過本節提供的觀察標準，作為未來模型學習行為分析的依據。

為了進一步優化效能，未來工作可朝以下幾個方向進行：首先，MCTS 中的搜尋節點應更系統性地使用 Hash Map 結構儲存已擴展過的節點資訊，包含該棋盤狀態對應的 policy 分布 (π) 與 value 值 (v)。目前缺乏這樣的快取機制，會導致相同狀態重複呼叫神經網路進行推論，不僅效率低落，也浪費 GPU 推理時間。其次，可引入「Virtual Loss」的概念於 MCTS 平行模擬中。當多個執行緒同時對同一節點進行模擬展開時，容易造成「重複擴展相同節點」的情況，進而浪費計算資源，並造成錯誤的策略探索。Virtual Loss 機制透過在展開節點時臨時加入虛擬的負面回饋，使其他執行緒避開該節點，達到資源分散與擴展分工的目的。

此外，為進一步解決個人電腦資源不足的問題，我們打算加入 client-server 架構，將神經網路推論流程與 MCTS 模擬流程分離。

MCTS 客戶端可集中於多執行緒模擬與節點選擇，而伺服器端則專注於批次處理多筆棋盤狀態的神經網路推論。讓此專案即使在單台個人電腦上亦有潛力實現高效率訓練。

本專題建立了一個基礎完整、可運作的六子棋強化學習程式。未來期望透過對 MCTS 的優化，有望使本專案能在資源有限的情況下持續增進學習表現。

REFERENCES

- [1] 楊子頤, "應用 AlphaZero 於六子棋", 國立交通大學多媒體工程研究所, 2020
- [2] 林育璋, "使用 KataGo 方法及迫招空間搜尋提升 Alphazero 在六子棋的訓練成效", 2023
- [3] 黃士豪, "利用有利條件訓練神經網路-以六子棋為例", 國立臺灣師範大學資訊工程研究所碩士論文, 2019。
- [4] 蔣秉璽, "六子棋程式強度改進之研究", 國立交通大學資訊科學與工程研究所碩士論文, 2015。
- [5] 李榮欽, "六子棋上的自動詰棋產生系統", 國立交通大學資訊科學與工程研究所論文, 2008。
- [6] 陳志宏, "在下棋與訓練階段改進 AlphaZero 演算法", 國立台灣師範大學資訊工程研究所博士論文, 2021。
- [7] D. Silver et al., "Mastering the game of go with deep neural networks and tree search", Nature, vol. 529, no. 7587, pp. 484-489, 2016.
- [8] D. Silver et al., "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm"
- [9] J. Czech, P. Korus and K. Kersting, "Monte-Carlo Graph Search for AlphaZero," Dec. 2020, Online Available: <https://arxiv.org/abs/2012.11045>.